Projet 3A - Département GIMA - Parcours IM

Étude des Réseaux de Neurones sur Graphes (GNN)

SPEISMANN Matthieu

Ecole Nationale Supérieure des Mines de Nancy

Année scolaire 2025-2026

Table des matières

Table des matières 2

Remerciements 3

Introduction 4
- I Intérêts de l'étude . . . 4
- II Rappels sur les Graphes . . . 4

I Découverte des GNN 6
- I Choisir la bonne représentation pour les graphes . . . 7
- II Construction d'un GNN simple . . . 9
  - II-A Le modèle de base . . . 9
  - II-B Les opérations de pooling . . . 10
  - II-C Utilisation de la représentation globale : Le Master Node . . . 12

II Les Graph Convolutional Networks (GCN) 13
- I Les Convolutions Polynomiales . . . 14
  - I-A Le filtre Laplacien . . . 14
  - I-B Les polynômes du Laplacien . . . 16
  - I-C Le ChebNet . . . 18
  - I-D Utilisation dans un Graph Convolution Network (GCN) . . . 19
- II La Convolution Spectrale . . . 20
  - II-A Définition de la représentation spectrale . . . 20
  - II-B La troncature en représentation spectrale . . . 21
  - II-C Utilisation dans un Graph Convolutional Network (GCN) . . . 22

III Application des GNN 23
- I Un problème de classification de graphes . . . 24
  - I-A Présentation du jeu de données . . . 24
  - I-B Modélisation informatique . . . 25
  - I-C Définition du modèle par GNN . . . 26
  - I-D Utilisation du modèle et analyse des résultats . . . 28
  - I-E Résumé de l'influence des hyperparamètres . . . 34
- II Extension à un dataset plus grand . . . 35
  - II-A Présentation du jeu de données . . . 35
  - II-B Modélisation informatique . . . 36
  - II-C Utilisation de modèles et analyse des résultats . . . 37
  - II-D Récapitulatif des résultats . . . 44

Conclusion 45

Bibliographie 47

Remerciements

Avant toute chose, je souhaiterais remercier chaleureusement Monsieur Frédéric Sur pour son suivi et sa disponibilité ainsi que pour tous ses précieux conseils et les nombreuses indications qu'il a su me fournir pour la bonne réalisation de ce projet.

Je tiens également à remercier Madame Sandie Ferrigno et Monsieur Yannick Privat du département IM pour leur rôle important dans l'organisation des projets de 3ème année au sein du département.

Introduction

I Intérêts de l'étude

Le but de ce projet est d'étudier, comprendre et utiliser des Graph Neural Networks (GNN). L'objectif des GNN est d'adapter les méthodes de réseaux de neurones pour réaliser des tâches de régression, de classification et/ou de prédiction sur des données structurées sous la forme de graphes.

Ce sujet est particulièrement intéressant car beaucoup de situations concrètes peuvent être représentées avec des données sous forme de graphes, parmi lesquels :

- Les réseaux sociaux
- Les molécules chimiques
- Les chaines logistiques d'approvisionnement, de production et de livraison.

II Rappels sur les Graphes

Définition : Graphe

Un graphe G est défini par :

- Un ensemble de nœuds (ou sommets) $V = \{v_1, v_2, \ldots, v_n\}$
- Un ensemble d'arêtes (ou liens) $E \subseteq \{\{u, v\}, u \in V, v \in V\}$

On dira que le graphe est orienté si les arêtes sont des couples ordonnés (u, v).

On pourra de plus pondérer les nœuds, les arêtes et/ou le graphe entier en leur associant des valeurs scalaires ou vectorielles.

Cela peut représenter, par exemple :

- **Pour les nœuds** : des caractéristiques propres à chaque nœud (âge, localisation, etc. dans le cas d'un réseau social)
- **Pour les arêtes** : des caractéristiques propres aux relations entre deux nœuds (force de la relation, distance géographique, etc.)
- **Pour le graphe entier** : des caractéristiques globales du graphe (type de réseau social, secteur d'activité dans le cas d'une chaine logistique, etc.)

Pour la suite de ce document, il est important de rappeler quelques notions de base sur les graphes.

Définition : Distance dans un graphe

La distance entre deux nœuds u et v d'un graphe G, notée $\text{dist}_G(u, v)$ est définie comme le nombre minimum d'arêtes à parcourir pour aller de u à v.

Degré d'un nœud

Le degré d'un nœud v dans un graphe est défini comme le nombre d'arêtes connectées à ce nœud.

Chapitre I

Découverte des GNN

L'ensemble de ce chapitre se base sur la lecture de l'article *A gentle Introduction to Graph Neural Networks* [1].

Les GNN ont pour objectif de généraliser les tâches de prédiction (classification et régression par exemple) des réseaux de neurones « standards » sur les graphes ou leurs composants. On classera donc les tâches selon qu'elles sont appliquées sur :

- **Les nœuds (node-level tasks)** : prédiction des valeurs d'un nœud, classification des nœuds, etc.
- **Les arêtes (edge-level tasks)** : prédiction de la présence (ou non) d'une arête, établir la relation entre les nœuds (classification d'arêtes), etc.
- **Le graphe entier (global-level tasks)** : le plus souvent une classification du graphe entier.

Dans un réseau de neurones standard, les données en entrée n'ont pas de structure particulière. Chaque donnée (i.e. la valeur à prédire et ses descripteurs) peut être lue et traitée indépendamment des autres.

La structure des données sous forme de graphe est alors l'élément différentiant par rapport aux approches habituelles et représente donc la principale difficulté pour travailler avec un réseau de neurones. En effet, la connectivité des nœuds entre eux doit être prise en compte pour que le modèle puisse exploiter cette information structurelle, et fournir des résultats satisfaisants qui exploitent cette particularité.

I Choisir la bonne représentation pour les graphes

Pour représenter les données en graphes de manière satisfaisante pour un réseau de neurones, une solution naïve semble être d'utiliser **la matrice d'adjacence**.

Définition : Matrice d'adjacence

Une matrice d'adjacence A d'un graphe $G = (V, E)$ est une matrice carrée de taille $|V| \times |V|$ telle que :
(|.| correspondant au cardinal)

$$A_{ij} = \begin{cases} 1 & \text{si } (v_i, v_j) \in E \\ 0 & \text{sinon} \end{cases}$$

Cette représentation intuitive a cela dit deux défauts problématiques dans notre cas :

1. La matrice d'adjacence stocke des valeurs pour l'ensemble des arêtes possibles, soit, dans un graphe à n nœuds, n^2 valeurs.
   Cependant, dans la majorité des cas, les graphes sont creux (sparse), c'est-à-dire qu'ils contiennent beaucoup moins d'arêtes que le nombre maximum possible.
   Cela signifie que la matrice d'adjacence contiendra beaucoup de zéros, ce qui est inefficace en termes de mémoire et de calcul, car ce sont bien les arêtes présentes qui portent l'information.
2. Dans une matrice d'adjacence, rien ne nous interdit de permuter les nœuds du graphe. Il existe donc en réalité $n!$ matrices d'adjacence équivalentes pour un même graphe.
   Le problème, c'est que toutes ces représentations équivalentes d'un même graphe, après passage dans un réseau de neurones, ne garantissent pas de fournir un résultat identique. Cela signifie qu'il serait possible d'obtenir des résultats différents pour un même graphe, simplement en permutant les nœuds dans la matrice d'adjacence. *La figure I.1 illustre cette difficulté.*

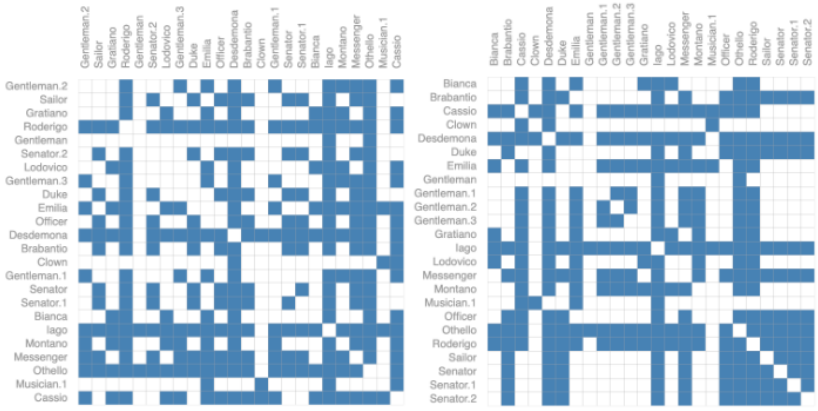


FIGURE I.1 – *Exemple de deux matrices d'adjacence représentant un même graphe. [1]*

Pour résoudre ces deux problèmes, on utilise donc une autre représentation, basée sur la **liste d'adjacence**.

Représentation par liste d'adjacence

- La liste V_0 des valeurs prises par les nœuds
- La liste d'adjacence L, qui est la liste des arêtes présentes dans le graphe
- La liste E_0 des valeurs prises par les arêtes
- Une valeur U_0 représentant globalement le graphe entier (*voir la sous-section II-C — Utilisation de la représentation globale : Le Master Node — pour son utilisation*)

La figure I.2 représente cette approche sur un exemple.

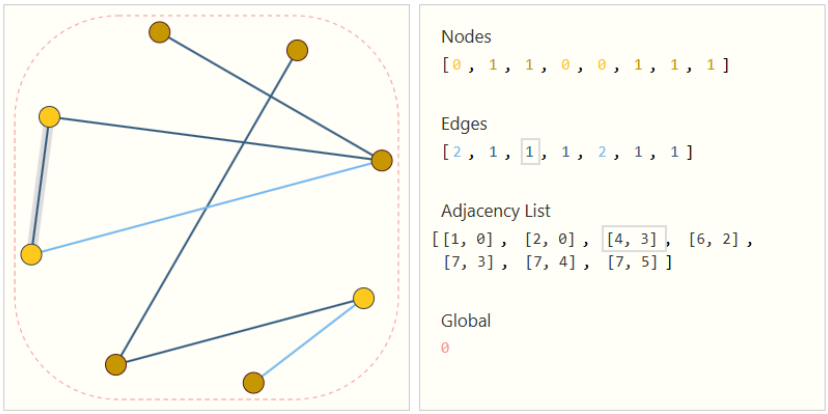


FIGURE I.2 – *Exemple de représentation d'un graphe par liste d'adjacence [1]*

Dans l'exemple ci-dessus, les valeurs prises par les nœuds, les arêtes et le graphe global sont scalaires, mais la représentation se généralise aux valeurs vectorielles.

L'utilisation de la liste d'adjacence permet de ne stocker que les arêtes présentes dans le graphe, ce qui est plus efficace en termes de mémoire et de calcul.

De plus, les conditions sur l'ordre des nœuds et des arêtes permettent d'assurer une invariance de la représentation du graphe par permutation des nœuds.

II Construction d'un GNN simple

II-A Le modèle de base

En utilisant la représentation précédemment définie, on peut construire un premier modèle de GNN simple.

En notant U_n, V_n et E_n les valeurs respectives du graphe global, des arêtes et des nœuds à l'étape n du réseau de neurones, on a alors :

- $U_{n+1} = f(U_n)$
- $V_{n+1} = g(V_n)$
- $E_{n+1} = h(E_n)$

Où f, g et h sont des *update functions* qui peuvent être chacune un réseau de neurones standard, ou toute autre fonction souhaitée. La figure I.3 propose une représentation de cette démarche.

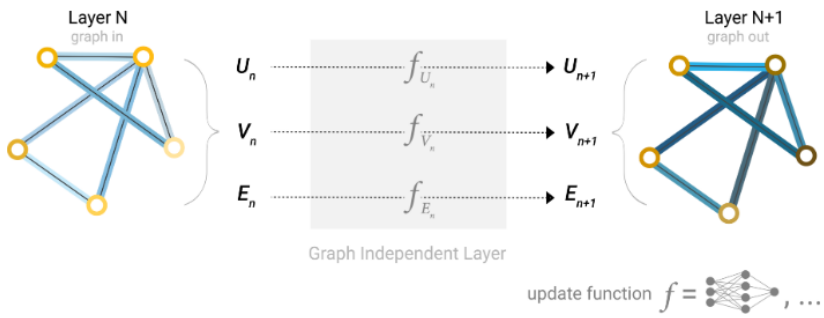


FIGURE I.3 – *Schéma d'un bloc GNN [1]*

Cette démarche forme alors un bloc GNN que l'on peut alors répéter à partir de la représentation initiale du graphe (U_0, V_0, E_0) pour calculer les représentations finales (U_n, V_n, E_n).

Ensuite, on va chercher à faire des prédictions à partir de la sortie des blocs GNN. Pour cela, on applique simplement une fonction de prédiction C sur les valeurs de sortie du bloc GNN, en choisissant l'ensemble U_n, V_n ou E_n selon la tâche à réaliser, comme illustré sur la figure I.4 pour une tâche de classification de nœuds.

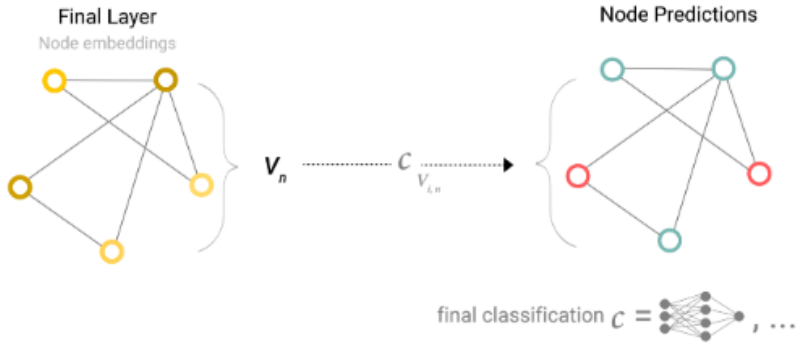


FIGURE I.4 – *Schéma d'une tâche de classification de nœuds [1]*

À ce stade, il est important de noter que ce premier modèle de GNN ne prend pas du tout en compte la connectivité du graphe, puisque nous ne nous sommes pas encore servis de la liste d'adjacence.

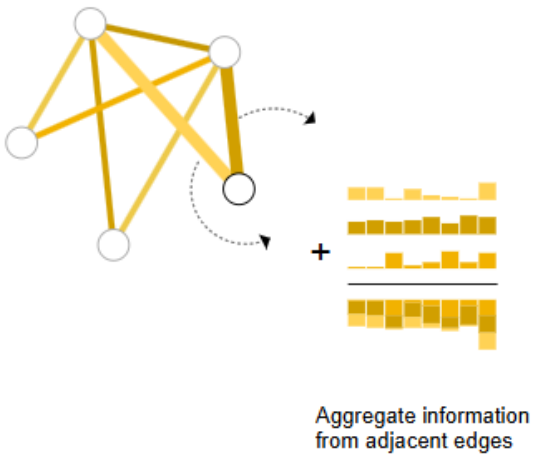
II-B Les opérations de pooling

Pour prendre en compte la connectivité du graphe, on va introduire une opération clé des GNN : le **pooling**.

Définition : Le pooling

Le pooling est une opération qui agrège les informations des nœuds et/ou arêtes voisins d'un nœud et/ou arêtes donné pour mettre à jour la valeur de cet élément en prenant en compte son contexte local dans le graphe.

Cette opération peut être réalisée de différentes manières, telles que la somme, la moyenne, le maximum, ou des méthodes plus complexes, comme avec des réseaux d'attention.



Sur la figure I.5 ci-contre, on effectue une opération de pooling sur le nœud sélectionné (en gras), en agrégeant les informations des arêtes adjacentes pour mettre à jour la valeur du nœud.

Cela dit, les représentations des nœuds et des arêtes, ne sont pas nécessairement identiques, en particulier la taille du vecteur qui les représente. Pour cette raison, il faut s'intéresser au conditionnement des pooling.

FIGURE I.5 – *Pooling sur un nœud [1]*

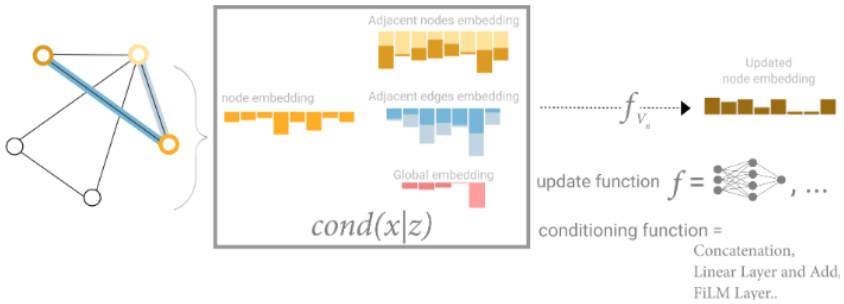


FIGURE I.6 – *Le conditionnement des pooling [1]*

Pour conditionner les poolings, et donc gérer les différences de taille entre les représentations des nœuds et des arêtes, on utilise des fonctions de conditionnement qui permettent d'adapter les valeurs avant l'agrégation. Celles-ci peuvent être des concaténations, des couches linéaires de projections (Linear Mapping Layers) (à apprendre avec un autre réseau par exemple) ou d'autres fonctions d'adaptation.
Cette problématique est illustrée sur la figure I.6 ci-dessus.

Intéressons-nous maintenant à deux usages du pooling dans les GNN :
Le pooling en fin de réseau, juste avant la prédiction :

Dans ce cas, à partir des informations finales (U_n, V_n, E_n) en sortie des blocs GNN, on applique une opération de pooling pour obtenir une nouvelle représentation agrégée du graphe avant de faire la prédiction. Une représentation de ce pooling est proposée par la figure I.7.

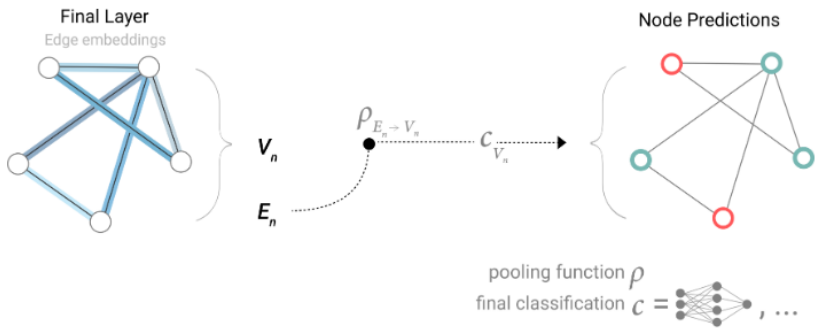


FIGURE I.7 – *Schéma d'un exemple sur une tâche de classification de nœuds [1]*

Le pooling dans les blocs GNN - le message-passing :

On peut aussi faire le pooling à l'intérieur des blocs GNN, pour prendre en compte la connectivité du graphe à chaque étape de mise à jour des valeurs (U_n, V_n, E_n). On parle alors de *message-passing* et cela permet de propager l'information à travers le graphe, en mettant à jour les valeurs des nœuds et des arêtes en fonction de leurs voisins. C'est donc ici que la connectivité du graphe sert.

On voit sur la figure I.8 ci-dessous que, pour effectuer le message-passing pour atteindre les nœuds voisins, de par la représentation utilisée, on doit en fait d'abord faire un message-passing des nœuds vers les arêtes, puis des arêtes vers les nœuds.

De la même manière, on peut faire un message-passing sur les arêtes en utilisant les nœuds comme intermédiaires, et en superposant les deux approches, on obtient un message-passing local complet, représenté par la figure I.9.

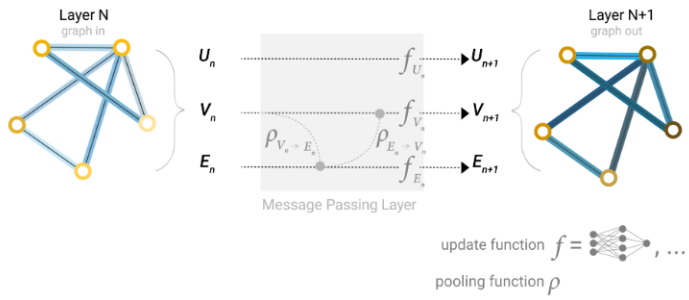


FIGURE I.8 – *message-passing sur les nœuds [1]*

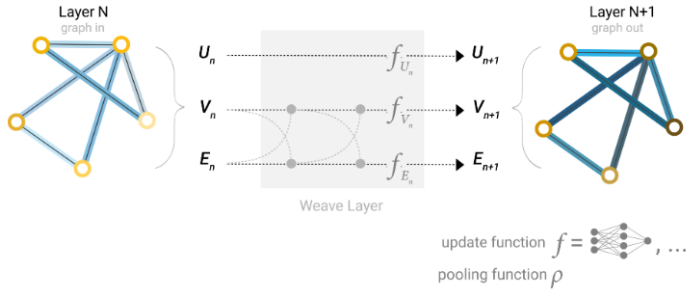


FIGURE I.9 – *message-passing local complet [1]*

Distance de réceptivité du message-passing

k étapes de message-passing permettent à chaque nœud (ou arête) d'incorporer l'information de son k-hop voisinage, c'est-à-dire des nœuds (ou arêtes) situés à une distance maximale de k.

Cela signifie que, pour capturer des informations plus globales du graphe, il est nécessaire d'augmenter le nombre de répétitions de message-passing effectuées dans le GNN. Mais pour les très grands graphes (par exemple les réseaux sociaux), le nombre à considérer est bien trop important pour envisager de le faire de manière efficace.

II-C Utilisation de la représentation globale : Le Master Node

L'utilisation de la représentation globale du graphe, aussi appelée **Master Node**, permet de contourner la limitation du message-passing pour capturer des informations globales du graphe.

Pour ce faire, on fait un pooling du Master Node vers les nœuds et les arêtes, puis un pooling des nœuds et des arêtes vers le Master Node. Cela permet au Master Node de collecter des informations de l'ensemble du graphe, et de les redistribuer aux nœuds et arêtes à chaque étape, facilitant ainsi la propagation d'informations globales à travers le graphe.
La figure I.10 illustre cette approche de la même manière que pour les exemples précédents.

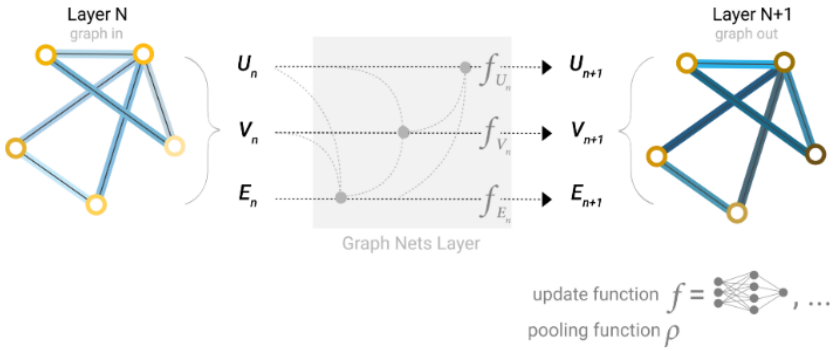


FIGURE I.10 – *Schéma d'une utilisation du Master Node dans un bloc GNN [1]*

Chapitre II

Les Graph Convolutional Networks (GCN)

L'ensemble de ce chapitre se base sur la lecture de l'article *Understanding Convolutions on Graphs* [2].

Un cas particulier de GNN très utilisé est celui des **Graph Convolutional Networks (GCN)**, dont l'objectif principal est de généraliser les réseaux de convolutions aux graphes.

On rappelle que les opérations de convolution sont des opérations locales, initialement pensées pour les images, comme illustré par la figure II.1, permettent d'extraire des caractéristiques locales en appliquant des filtres sur des régions restreintes des données.

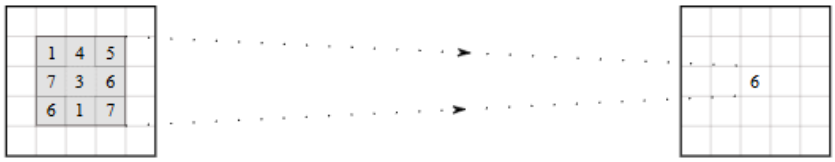


FIGURE II.1 – *Exemple de filtre de convolution sur les images [2]*

L'idée ici est donc de créer des filtres de convolution adaptés aux graphes, qui pourraient être utilisés dans des blocs GNN pour extraire des caractéristiques locales des graphes.
Une illustration de cet objectif est donnée à la figure II.2.

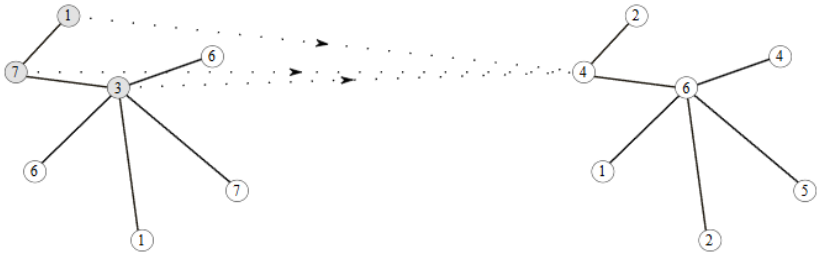


FIGURE II.2 – *Exemple de filtre de convolution sur les graphes [2]*

I Les Convolutions Polynomiales

I-A Le filtre Laplacien

Une première approche pour définir des filtres de convolution sur les graphes est d'utiliser le Laplacien du graphe.

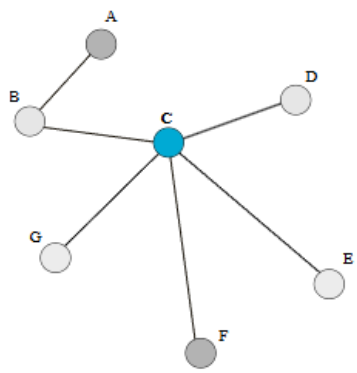
Définition : Le Laplacien d'un graphe

Le Laplacien L d'un graphe est défini comme :

$$L = D - A \tag{II.1}$$

Où D est la matrice de degré (matrice diagonale où chaque élément diagonal D_{ii} est égal au degré du nœud i) et A est la matrice d'adjacence du graphe.

La figure II.3 illustre cette définition sur un graphe d'exemple



$$\begin{array}{c|c} & \begin{array}{ccccccc} A & B & C & D & E & F & G \end{array} \\ \begin{array}{c} A \\ B \\ C \\ D \\ E \\ F \\ G \end{array} & \begin{bmatrix} 1 & -1 & & & & & \\ -1 & 2 & -1 & & & & \\ & -1 & 5 & -1 & -1 & -1 & -1 \\ & & -1 & 1 & & & \\ & & -1 & & 1 & & \\ & & -1 & & & 1 & \\ & & -1 & & & & 1 \end{bmatrix} \end{array}$$

FIGURE II.3 – *Exemple du Laplacien d'un graphe [2]*

En notant alors x les valeurs prises par les nœuds du graphe (dans l'ordre des nœuds utilisé dans A, D et L), on peut définir une opération de convolution sur le graphe en utilisant le Laplacien :

$$x' = Lx$$

Cette opération permet de propager l'information locale à travers le graphe, en mettant à jour les valeurs des nœuds en fonction de leurs voisins.

Propriété

La matrice L du Laplacien est **semi-définie positive** pour les graphes non orientés.

Preuve :

Soit $x \in \mathbb{R}^n$. $x^T L x = x^T (D - A) x = \sum_i d_i x_i^2 - \sum_{i,j} A_{ij} x_i x_j$

Or $d_i = \sum_j A_{ij}$, donc $\sum_i d_i x_i^2 = \sum_{i,j} A_{ij} x_i^2$.

Comme de plus, le graphe est non orienté, A est symétrique et donc $\sum_{i,j} A_{ij} x_i^2 = \frac{1}{2} \sum_{i,j} A_{ij} (x_i^2 + x_j^2)$.

Finalement $x^T L x = \frac{1}{2} \sum_{i,j} A_{i,j} (x_i^2 + x_j^2 - 2 x_i x_j) = \frac{1}{2} \sum (x_i - x_j)^2 \geqslant 0$

CQFD

I-B Les polynômes du Laplacien

On peut également élargir le noyau de convolution de cette approche. Pour cela, on utilise des polynômes du Laplacien pour définir des filtres de convolution entre des nœuds qui ne sont pas voisins directs.

En notant $P_w(L) = \sum_{i=0}^{d} w_i L^i$ un polynôme de degré d du Laplacien, on peut définir l'opération de convolution :

$$x' = P_w(L)x$$

avec w_i les coefficients du polynôme à apprendre.

Lien entre puissance de L et distance dans le graphe

Soit $i \in \mathbb{N}$. Soit u et v deux nœuds du graphe G.

Si u et v sont à une distance strictement supérieure à i dans le graphe G, alors la valeur $L^i_{uv} = 0$.

$$\text{dist}_G(v, u) > i \Rightarrow L^i_{uv} = 0$$

En particulier, la contraposée donne :

$$L^i_{uv} \neq 0 \Rightarrow \text{dist}_G(v, u) \leq i$$

Preuve :

Soit $i \in \mathbb{N}$. Soit u et v deux nœuds du graphe G tels que $\text{dist}_G(u, v) > i$.

Si $i = 1$, alors on a que $L_{uv} = 0$ car $u \neq v$ et qu'il n'existe donc pas d'arête entre u et v.
Si $i > 1$, on a, par répétition de l'expression du produit matriciel :

$$L^i_{uv} = \sum_k (L_{uk_1})(L_{k_1k_2})(L_{k_2k_3})...(L_{k_{i-1}v})$$

avec la somme prise sur l'ensemble des séquences de longueur $i - 1$: $(k_1, k_2, ..., k_{i-1})$.
Supposons maintenant par l'absurde que $L^i_{u,v} \neq 0$.
Alors, il existe une séquence $(k_1, k_2, ..., k_{i-1})$ telle que : $L_{uk_1} \neq 0, L_{k_1k_2} \neq 0, L_{k_2k_3} \neq 0, ..., L_{k_{i-1}v} \neq 0$
Cela implique l'existence d'un chemin de longueur inférieure ou égale à i entre u et v passant par les nœuds $k_1, k_2, ..., k_{i-1}$, ce qui contredit l'hypothèse que $\text{dist}_G(u, v) > i$.

CQFD.

Ce résultat signifie que que l'application d'un polynôme de degré d du Laplacien peut propager l'information jusqu'aux nœuds situés à une distance maximale de d dans le graphe.

Intéressons-nous à présent à une propriété importante des convolutions polynomiales, **l'équivariance par permutation** :

Définition : Permutation des nœuds d'un graphe

Soit un ordre fixé des nœuds du graphe.
On peut représenter une permutation de cet ordre de nœuds par une matrice de permutation P, qui est une matrice carrée binaire orthogonale, où chaque ligne et chaque colonne contient exactement un élément égal à 1.

Définition : Équivariance par permutation

Une opération f sur les graphes est dite **équivariante par permutation** si, pour toute permutation des nœuds représentée par une matrice de permutation P :

$$f(Px) = Pf(x)$$

Où x est une représentation quelconque du graphe (par exemple les valeurs des nœuds).

L'equivariance par permutation

Les convolutions polynomiales du Laplacien sont équivariantes par permutation.

Preuve :

Soit P une matrice de permutation.

En appliquant cette permutation aux éléments suivants du graphe :

- Les valeurs des nœuds x deviennent Px
- La matrice d'adjacence A devient PAP^T
- La matrice de degré D devient PDP^T
- Le Laplacien $L = D - A$ devient (par les deux points précédents) PLP^T
- Les puissances du Laplacien L^i deviennent $(PLP^T)^i = PL^iP^T$ grâce à l'orthogonalité de P

Finalement, en utilisant donc ici $f(x) = P_w(L)x$.

$$f(Px) = \sum_{i=0}^{d} w_i(PL^iP^T)(Px) = P\sum_{i=0}^{d} w_iL^ix = Pf(x)$$

CQFD

Ce résultat permet d'assurer que, réaliser une permutation des nœuds dans le graphe ne change pas le résultat de la convolution, et donc, dans le cas d'une utilisation en réseau, assure un résultat unique pour les différentes représentation d'un même graphe.

I-C Le ChebNet

Cette fois-ci, on définit $P_w(L) = \sum_{i=1}^{d} w_i T_i(\tilde{L})$ où

$$\begin{cases} \tilde{L} = \dfrac{2L}{\lambda_{max}} \text{ est le Laplacien normalisé où } \lambda_{max} \text{ est la valeur propre maximale de } L \\ T_i \text{ est le polynôme de Tchebychev de la première espèce, de degré } i \end{cases}$$

L'opération de convolution est toujours définie comme :

$$x' = P_w(L)x$$

avec w_i les coefficients du polynôme à apprendre.

Explicitons à présent les motivations derrière ces choix :

1. En notant que L est semi-définie positive, toutes ses valeurs propres sont positives. En particulier, si $\lambda_{\max} > 1$ les puissance de L vont exploser, ce qui est problématique pour la stabilité numérique.
   Cette normalisation assure donc que toutes les valeurs propres de $\tilde{L}$ soient désormais dans l'intervalle $[-1, 1]$, et les puissances de L restent bornées.
2. L'utilisation des polynômes de Tchebychev permet d'obtenir une base orthogonale pour les polynômes, ce qui améliore la stabilité numérique.

Rappel sur les polynômes de Tchebychev de la première espèce

Les polynômes de Tchebychev de la première espèce sont les polynômes orthogonaux vérifiant la propriété suivante :

$$T_n(\cos\theta) = \cos(n\theta)$$

On les obtient donc avec la relation de récurrence :

$$\begin{cases} T_0 = 1 \\ T_1 = X \\ T_{n+1} = 2XT_n - T_{n-1} \end{cases}$$

Il est donc clair que $\forall n \in \mathbb{N}, T_n$ est un polynôme de degré n.

Propriétés du ChebNet

Le ChebNet, défini ici, hérite des propriétés des convolutions polynomiales :

- Il est capable de capturer des informations locales jusqu'à une distance de réceptivité égale au degré du polynôme utilisé.
- Il est équivariant par permutation des nœuds du graphe.

I-D Utilisation dans un Graph Convolution Network (GCN)

L'utilisation générale des convolutions polynomiales ou Chebnet dans un GCN est relativement simple, en calculant les représentations des valeurs des nœuds à chaque couche par :

On note h^0 le vecteur contenant les valeurs initiales des nœuds (scalaires ici)

Puis, $\forall k \in \mathbb{N}$:

1. Définition du polynôme de convolution utilisé à la couche k (représenté matriciellement) :
$$p^{(k)} = P_{w^{(k)}}(L)$$
2. Application de la convolution :
$$g^{(k)} = p^{(k)} h^{(k-1)}$$
3. Application d'une fonction d'activation σ pour la non-linéarité :
$$h^{(k)} = \sigma(g^{(k)})$$

L'approche qui suit est utilisée et détaillée dans l'article original de Kipf et Welling *Semi-Supervised Classification with Graph Convolutional Networks* (2017). [3]

- On développe le calcul qui précède, avec un polynôme du Laplacien $P_w = \sum_{i=0}^{d} w_i L^i$, on obtient :
$$h^{(k)} = \sigma\left(\left(\sum_{i=0}^{d} w_i L^i\right) h^{(k-1)}\right)$$
Ici, les poids w_i impactent chacun une puissance différente du Laplacien, et donc une distance de réceptivité différente, sans mélange.
- Or, pour l'apprentissage, il peut être intéressant de mélanger les informations provenant de différentes distances de réceptivité. Pour cela, en fixant $w_i = w$ pour tout i, on obtient :
$$h^{(k)} = \sigma\left(w\left(\sum_{i=0}^{d} L^i\right) h^{(k-1)}\right)$$
- Enfin, on peut généraliser cette approche aux valeurs vectorielles des nœuds, en notant $H^{(k)} \in \mathbb{R}^{n \times d_k}$ la matrice des représentations des nœuds à la couche k, avec d_k le nombre de features (i.e. la taille du vecteur représentant chaque nœud) à la couche k.

On a alors la forme finale souhaitée :
$$H^{(k)} = \sigma\left(\left(\sum_{i=0}^{d} L^i\right) H^{(k-1)} W^{(k)}\right) \tag{II.2}$$
Ce qui s'écrit de manière équivalente, pour chaque nœud v, en intégrant les valeurs de L^i_{uv} dans W :
$$h_v^{(k)} = \sigma\left(W^{(k)}\left(\sum_{\mathrm{Dist}_G(u,v) \leq d} h_u^{(k-1)}\right)\right) \tag{II.3}$$

II La Convolution Spectrale

Toutes les méthodes de convolutions sur les graphes vues précédemment sont des méthodes locales. Pour faire le lien avec le chapitre I, ces convolutions peuvent être vues comme des étapes de message-passing locaux à chaque itération des blocs GNN.

Comme vu dans le chapitre précédent, ces méthodes ne permettent donc pas de capturer efficacement des informations globales du graphe. Pour cela, on va donc maintenant s'intéresser aux convolutions spectrales.

II-A Définition de la représentation spectrale

Il faut commencer par normaliser x tel que $\sum_{i=1}^{n} x_i^2 = 1$.

Ensuite, d'après le théorème spectral, le Laplacien L d'un graphe étant réel symétrique, il est diagonalisable.

Il existe donc U une matrice contenant les vecteurs propres, pris orthonormaux entre eux ($UU^T = U^TU = I$)

et $\Lambda = \begin{pmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & & \vdots \\ \vdots & & \ddots & 0 \\ 0 & \cdots & 0 & \lambda_n \end{pmatrix}_{\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n}$ la matrice diagonale des valeurs propres de L, telle que : $L = U\Lambda U^T$

La matrice U définit donc une base orthonormée qui peut être utilisée pour effectuer un changement de base des valeurs des nœuds x du graphe, appelé **transformée de Fourier sur les graphes** :

$$\hat{x} = U^T x = \sum_{i=1}^{n} x_i u_i^T$$

La transformée de Fourier inverse est-elle donnée par :

$$x = U\hat{x} = \sum_{i=1}^{n} \hat{x}_i u_i$$

Le changement de base permet bien de prendre en compte la structure du graphe globalement, car les vecteurs propres u_i sont des fonctions définies sur l'ensemble des nœuds du graphe.

On définit le quotient de Rayleigh par $R_L(x) = \dfrac{x^T L x}{x^T x} = \sum_{(i,j)\in E} (x_i - x_j)^2$
(*par définition de L et car x est normalisé, $x^T x = 1$*)

Par analogie avec la théorie de Fourier, on peut alors interpréter les valeurs propres les plus faibles comme celles associées aux basses fréquences dans le graphe, liées aux différences de valeurs "lisses" entre des nœuds éloignés dans le graphe, c'est-à-dire des tendances globales.

A l'inverse, les valeurs propres les plus élevées sont associées aux hautes fréquences, liées aux différences de valeurs entre des nœuds proches dans le graphe, captant alors des différences moins "lisses", des détails très locaux.

II-B La troncature en représentation spectrale

Cependant, cette approche présente un inconvénient majeur : le calcul de la transformée de Fourier nécessite la diagonalisation du Laplacien, ce qui est coûteux en temps de calcul pour les grands graphes.

Or, en notant $\hat{x}_m = \text{Tronquer}_m(\hat{x}) = \text{Tronquer}_m(U^T x)$, la troncature aux m premiers coefficients de Fourier (en fixant à 0 les suivants). Cette opération de troncature est en fait équivalente à utiliser uniquement les m premiers vecteurs propres U_m, associés aux m valeurs propres les plus faibles, ce qui représente alors surtout les informations globales du graphe.

On peut alors définir une convolution spectrale tronquée, en utilisant uniquement les m premiers vecteurs propres :

$$\hat{x} = U_m^T x$$

et son inverse, pour revenir en représentation spatiale :

$$x_m = U_m \hat{x}_m$$

Cette approche permet de réduire considérablement le coût de calcul puisqu'il est plus rapide de déterminer m vecteurs propres que tous. De plus, on capture bien les caractéristiques globales du graphe en ne perdant que peu d'information sur les données.

Les exemples des figures II.4 et II.5 ci dessous représente cette troncature sur des images de 2000 pixels, traités comme des graphes (les pixels sont des nœuds interconnectés avec leurs voisins), sur lesquels on a appliqué une convolution spectrale.

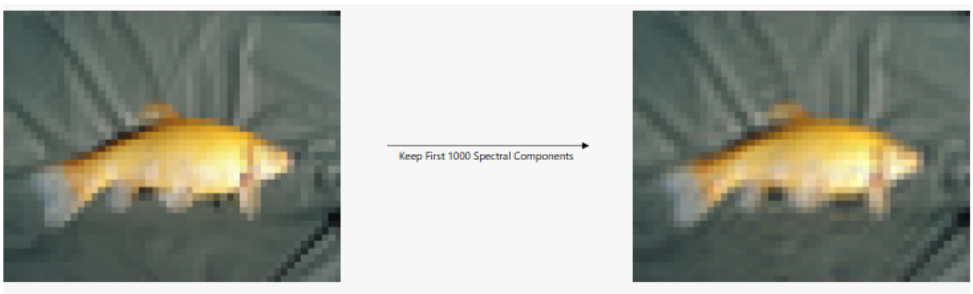


FIGURE II.4 – *Troncature avec 1000 vecteurs propres [2]*

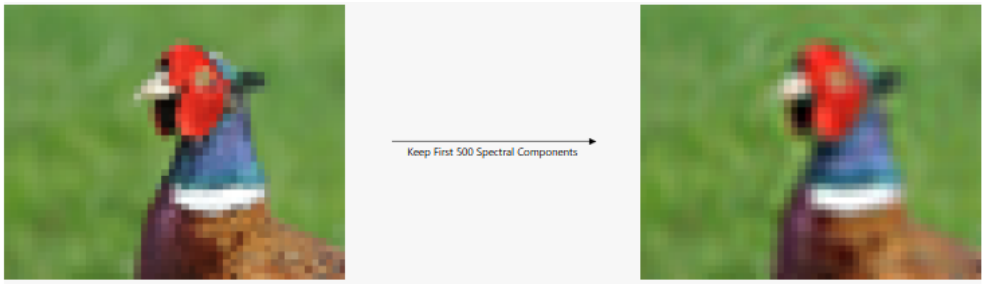


FIGURE II.5 – *Troncature avec 500 vecteurs propres [2]*

II-C Utilisation dans un Graph Convolutional Network (GCN)

L'utilisation de la convolution spectrale (éventuellement tronquée) dans un GCN est ensuite similaire à l'utilisation des convolutions polynomiales.

On note H^0 la matrice contenant les vecteurs de représentation de chaque nœud.

Puis, $\forall k \in \mathbb{N}$:

1. Passage en représentation spectrale, éventuellement tronquée
$$\hat{H}^{(k-1)} = U_m^T H^{(k-1)}$$
2. Ajout des poids d'apprentissage (appris par le réseau) par produit de Hadamard (i.e. le produit terme à terme des matrices) :
$$\hat{G}^{(k)} = W^{(k)} \odot H^{(k-1)}$$

   Résultat admis :
   C'est ici qu'a lieu la convolution, puisque la transformée de Fourier effectuée à l'étape 1 transforme les produits de Hadamard en convolutions dans le domaine spatial.

3. Retour en représentation spatiale :
$$G^{(k)} = U_m \hat{G}^{(k)}$$
4. Application d'une fonction d'activation σ pour la non-linéarité :
$$H^{(k)} = \sigma(G^{(k)})$$

Finalement, tout cela s'écrit de manière compacte :

$$H^{(k)} = \sigma\left(U_m\left(W^{(k)} \odot \left(U_m^T H^{(k-1)}\right)\right)\right) \tag{II.4}$$

Chapitre III

Application des GNN

L'objectif de ce chapitre est de présenter une application pratique des GNN pour démontrer l'efficacité de leur utilisation sur des données réelles, tout en faisant le lien avec les chapitres précédents en analysant l'influence des paramètres propres aux GNN sur les résultats obtenus.

Pour réaliser ces différents objectifs, on utilisera la bibliothèque Python `TensorFlow GNN` qui permet de construire et d'entraîner facilement des GNN. Celle-ci est avantageuse car elle est construite à partir de `TensorFlow`, une bibliothèque populaire pour la construction de réseau de neurones.

I Un problème de classification de graphes

L'ensemble de ce qui suit est tiré d'un exemple inclu dans la documentation officielle du module `TensorFlow GNN`. [4].

I-A Présentation du jeu de données

Le jeu de données utilisé est **Mutag**, un ensemble de données couramment utilisé pour évaluer les performances des algorithmes d'apprentissage automatique sur des graphes. Il contient 188 graphes représentant des molécules chimiques, où chaque graphe correspond à une molécule.

L'objectif est de classer les molécules pour determiner si elles sont mutagènes (c'est-à-dire susceptibles de provoquer des mutations génétiques chez les êtres vivants) ou non.

Ici, dans le jeu de données d'entrainement, les molécules possèdent en moyenne 18.03 atomes et 96 sur 150 (64%) sont mutagènes, et donc celui de validation, les molécules ont, en moyenne 17.55 atomes et 29 sur 38 (76.32%) sont mutagènes.

Cet écart dans la répartition des classes entre les deux ensembles est un point important à prendre en compte lors de l'évaluation des performances du modèle.

Les nœuds représentent les différents types d'atomes parmi :

- Le carbone (C)
- L'azote (N)
- L'oxygène (O)
- Le fluor (F)
- L'iode (I)
- Le chlore (Cl)
- Le brome (Br)

Les arêtes représentent les différents types de liaisons entre les atomes :

- Liaison simple
- Liaison double
- Liaison triple
- Liaison aromatique *(Ce sont des liaisons particulières présentes dans les molécules cycliques avec des propriétés aromatiques)*

Avec ces spécifications, il est possible de représenter les molécules sous forme de graphe avec le module `Networkx`, comme illustré sur la figure III.1 ci-dessous.

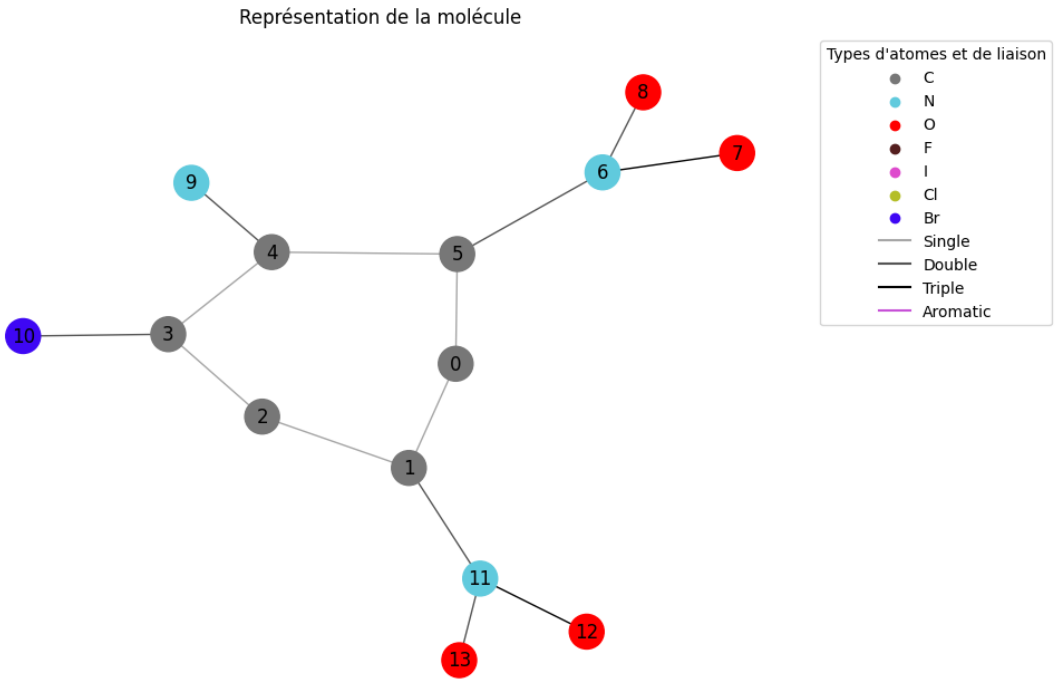


FIGURE III.1 – *Représentation d'une molécule du jeu de données Mutag avec* `Networkx`

I-B Modélisation informatique

On stocke pour chaque graphe `g` :

- Les features de chaque nœud (qui représentent les types d'atomes) sont stockés sous forme de vecteurs d'encodage one-hot dans l'attribut `node_sets['atoms'].features[tfgnn.HIDDEN_STATE]`
- Les arêtes du graphe sont stockées dans les attributs `edge_sets['bonds'].adjacency.source` et `edge_sets['bonds'].adjacency.target`, représentant respectivement les indices des nœuds sources et cibles de chaque arête.
- Les features de chaque arête (qui représentent les types de liaisons) sont stockés sous forme de vecteurs d'encodage one-hot dans l'attribut `edge_sets['bonds'].features[tfgnn.HIDDEN_STATE]`, respectivement à l'ordre des arêtes définies par les attributs d'adjacence.
- Le label de chaque graphe (mutagène ou non) est stocké dans une variable binaire `y`, séparé de `g`.

Pour les lots d'entraînement, on utilise la fonction `tfgnn.keras.layers.MergeBatchToComponents()` qui permet de fusionner plusieurs graphes en un seul objet de type graphe où les graphes individuels sont tous séparés.
Cela permet de traiter efficacement plusieurs graphes en parallèle lors de l'entraînement du modèle de GNN.

I-C Définition du modèle par GNN

Une fois ces différentes informations stockées, on peut définir les hyperparamètres du modèle de GNN qu'on va utiliser :

- `node_dim` : Les dimensions initiales des vecteurs de représentation des features des nœuds
- `edge_dim` : Les dimensions initiales des vecteurs de représentation des features des arêtes
- `message_dim` : Les dimensions des messages échangés entre les nœuds et les arêtes lors du message-passing
- `next_state_dim` : Les dimensions des vecteurs de représentation des nœuds et arêtes après chaque étape de message-passing
- `num_message_passing` : Le nombre d'étapes de message-passing effectuées dans le GNN
- `activation_function` : La fonction d'activation utilisée au cours de chaque étape de message-passing
- `dropout_rate` : Le taux de dropout appliqué pendant l'entraînement pour éviter le surapprentissage
- `l2_regularization` : Le coefficient de régularisation L2 appliqué aux poids du modèle pour éviter le surapprentissage

Une fois ces hyperparamètres définis, on peut construire le modèle de GNN avec ces étapes :

1. On commence par recevoir les entrées du graphe, c'est-à-dire les features des nœuds et des arêtes, qu'on va lier à une couche dense pour ajuster leurs dimensions aux dimensions souhaitées pour le modèle (`node_dim` et `edge_dim`).

```python
input_graph = tf.keras.layers.Input(type_spec=graph_tensor_spec)

# Lot de graphes d'entree:
graph = input_graph.merge_batch_to_components()

# Initialisation des dimensions des features des no\eu{}ds et aretes:
def set_initial_node_state(node_set, *, node_set_name):
    return tf.keras.layers.Dense(node_dim)(node_set[tfgnn.HIDDEN_STATE])

def set_initial_edge_state(edge_set, *, edge_set_name):
    return tf.keras.layers.Dense(edge_dim)(edge_set[tfgnn.HIDDEN_STATE])

graph = tfgnn.keras.layers.MapFeatures(
    node_sets_fn=set_initial_node_state, edge_sets_fn=set_initial_edge_state)(
        graph)
```

Listing III.1 – Initialisation des dimensions des features des nœuds et arêtes

2. On définit les couches denses utilisées plus tard par le message-passing, qui comprend la régularisation L2 et le dropout :

```
def dense(units, activation=activation_function)
    regularizer = tf.keras.regularizers.l2(l2_regularization)
    return tf.keras.Sequential([
        tf.keras.layers.Dense(
            units,
            activation=activation,
            kernel_regularizer=regularizer,
            bias_regularizer=regularizer),
        tf.keras.layers.Dropout(dropout_rate)
    ])
```

Listing III.2 – Définition des couches denses

3. Ensuite, on effectue le nombre d'étapes de message-passing définies par l'hyperparamètre `num_message_passing`, en utilisant des blocs de message-passing standard fournis par `Tensorflow GNN`.

```
for _ in range(num_message_passing):
    graph = tfgnn.keras.layers.GraphUpdate(
        node_sets={
            # Mise a jour des noeuds par utilisation de convolution:
            "atoms": tfgnn.keras.layers.NodeSetUpdate(
                {"bonds": tfgnn.keras.layers.SimpleConv(
                    sender_edge_feature=tfgnn.HIDDEN_STATE,
                    message_fn=dense(message_dim),
                    reduce_type="sum",
                    receiver_tag=tfgnn.TARGET)},
                tfgnn.keras.layers.NextStateFromConcat(dense(next_state_dim)))
                })(graph)
```

Listing III.3 – Étapes de message-passing

4. Enfin, on agrège les représentations finales des nœuds de chaque graphe du batch pour obtenir une représentation globale du graphe par pooling, et on utilise une couche dense finale pour prédire la classe (mutagène ou non) de chaque graphe du batch.

```
# Pooling des representations des noeuds pour obtenir une representation
    globale de chaque graphe du batch:
readout_features = tfgnn.keras.layers.Pool(
    tfgnn.CONTEXT, "mean", node_set_name="atoms")(graph)

# Couche de prediction linaire dense pour la classification binaire:
logits = tf.keras.layers.Dense(1)(readout_features)
```

Listing III.4 – Pooling et prédiction finale

Ensuite, on définit les métriques d'évaluation et la fonction de perte pour l'entraînement du modèle :

- La métrique utilisée pour la fonction de perte sera l'entropie croisée binaire.
- On s'intéressera également à la précision du modèle, qui mesure la proportion de prédictions correctes.

```
loss = tf.keras.losses.BinaryCrossentropy(from_logits=True)
metrics = [tf.keras.metrics.BinaryAccuracy(threshold=0.)]
```

Listing III.5 – Fonction de perte et metrique utilisées

I-D Utilisation du modèle et analyse des résultats

Modèle de référence

Pour notre premier essai, **qui servira de modèle de référence**, on utilise les hyperparamètres suivants :

- `node_dim = 16`
- `edge_dim = 16`
- `message_dim = 64`
- `next_state_dim = 64`
- `num_message_passing = 3`
- `activation_function = "relu"`
- `dropout_rate = 0.5`
- `l2_regularization = 5e-4`
- `batch_size = 16`
- `optimizer = Adam(learning_rate=0.01)`
- `steps_per_epoch = 10`

On obtient alors les courbes de pertes et de précision de la figure III.2, sur les données d'entrainement (Train) et de validation.

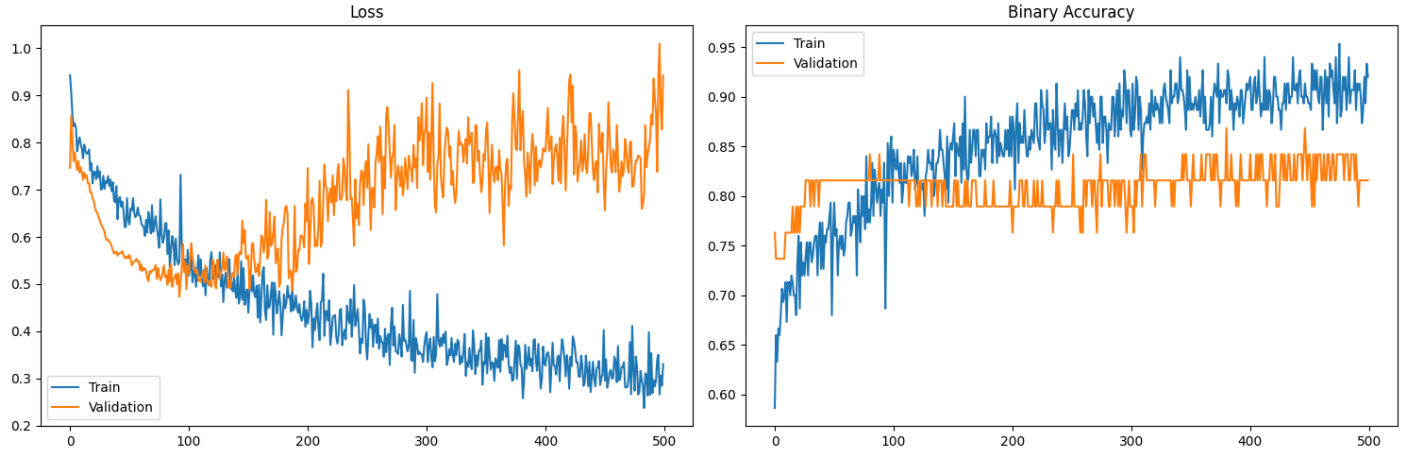


FIGURE III.2 – *Courbes de perte et de précision du modèle de référence*

On peut analyser cet entrainement de la façon suivante, en se basant sur les courbes et le code :

- Au-delà de la 92ème époque, le modèle est en surapprentissage. Cela se remarque car la perte de validation augmente à partir des époques suivante, avec une variance également croissante. La valeur minimale atteinte par la perte de validation,a cette époque est de 0.473
- La précision du modèle converge rapidement autour d'une valeur seuil.
  On utilisera comme référence la précision de validation atteinte à l'époque où la loss de validation minimale est atteinte.
  Ici c'est 81.6% atteint à l'époque 92
- Le nombre de paramètres du modèle est de 43665.
- L'entrainement de ces 500 époques a duré 1 minute et 6 secondes

Essayons à présent de modifier les hyperparamètres spécifiques aux GNN pour observer leur influence sur les résultats obtenus :

L'influence de `node_dim`

Essayons d'abord d'augmenter la dimension des vecteurs de représentation des nœuds initial, en passant de `node_dim = 16` à `node_dim = 64`.
Les résultats sont présentés sur la figure III.3 ci-dessous.

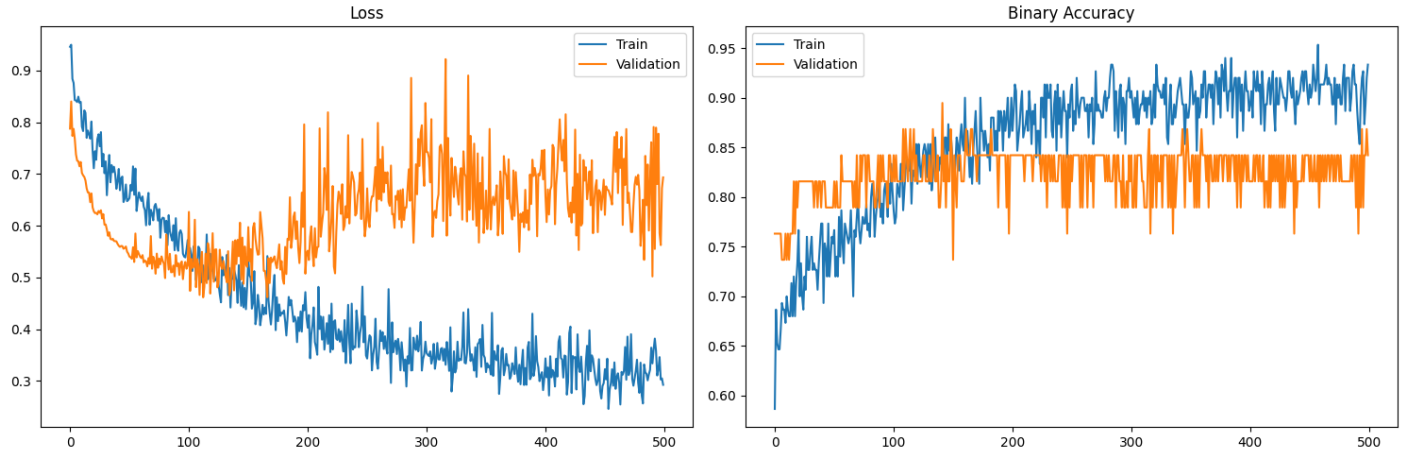


FIGURE III.3 – *Courbes de perte et de précision du modèle avec* `node_dim = 64`

- Au-delà de la 112ème époque, le modèle est en surapprentissage. Celui-ci se produit donc plus tardivement que pour le modèle de référence.
  La valeur minimale atteinte par la perte de validation est 0.461, très faiblement meilleure que le modèle de référence.
- A cette époque, la précision atteint 81.6%, soit identique au modèle de référence.
- Le nombre de paramètres du modèle a augmenté, atteignant 53265.
- L'entrainement de ces 500 époques a duré 1 minute et 8 secondes, une augmentation négligeable par rapport au modèle de référence.
  Ce résultat est attendu, car bien que l'augmentation de la dimension des nœuds augmente le nombre de paramètres du modèle, celui-ci reste encore de taille modeste.

L'influence de `edge_dim`

On va ensuite augmenter la dimension des vecteurs de représentation des arêtes initial, en passant de `edge_dim = 16` à `edge_dim = 64`.
Les résultats sont présentés sur la figure III.4 ci-dessous.

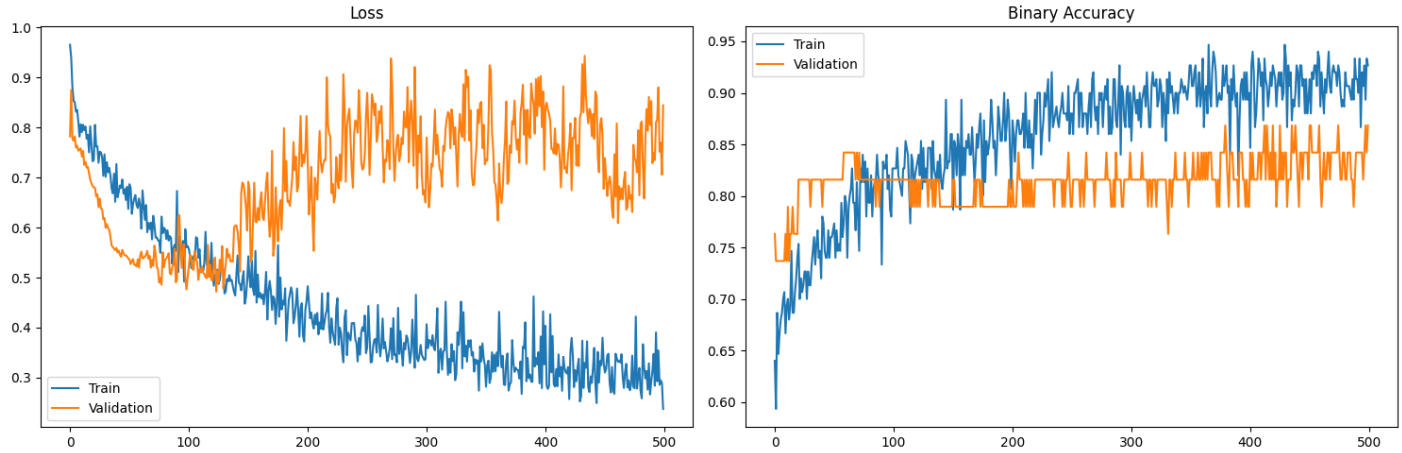


FIGURE III.4 – *Courbes de perte et de précision du modèle avec* `edge_dim = 64`

- Au-delà de la 123ème époque, le modèle est en surapprentissage. Celui-ci se produit donc plus tardivement que pour le modèle de référence.
  La valeur minimale atteinte par la perte de validation est 0.471, très faiblement meilleure que le modèle de référence.
- A cette époque, la précision atteint 81.6%, soit identique au modèle de référence.
- Le nombre de paramètres du modèle a augmenté, atteignant 53121.
- L'entrainement de ces 500 époques a duré 1 minute et 9 secondes, une augmentation négligeable par rapport au modèle de référence.
  Ce résultat est attendu, car bien que l'augmentation de la dimension des arêtes augmente le nombre de paramètres du modèle, celui-ci reste encore de taille modeste.

L'influence de `message_dim`

On va ensuite augmenter la dimension des vecteurs de représentation durant les étapes de message-passing, en passant de `message_dim = 64` à `edge_dim = 256`.
Les résultats sont présentés sur la figure III.5 ci-dessous.

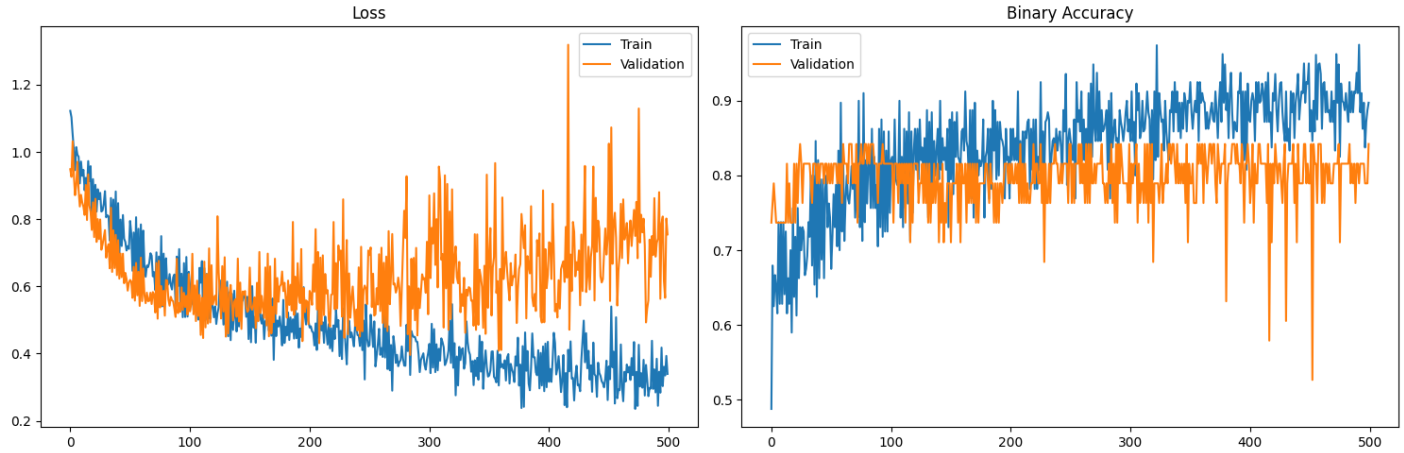


FIGURE III.5 – *Courbes de perte et de précision du modèle avec* `message_dim = 256`

- Au-delà de la 284ème époque, le modèle est en surapprentissage. Celui-ci se produit plus tardivement que pour le modèle de référence.
  La valeur minimale atteinte par la perte de validation est 0.396, bien meilleure que le modèle de référence.
- A cette époque, la précision atteint 84.2%, ce qui représente une amélioration sensible par rapport au modèle de référence.
- Le nombre de paramètres du modèle a considérablement augmenté, atteignant 145617.
  C'est cela qui justifie que le surapprentissage apparaisse plus tardivement, car il y a bien plus de poids à ajuster désormais.
- L'entrainement de ces 500 époques a duré 1 minute et 24 secondes, une augmentation significative par rapport au modèle de référence.
  Ce résultat est attendu, car bien que l'augmentation des dimensions de représentation dans le message-passing augmente fortement le nombre de paramètres du modèle, ce qui rend sensible l'augmentation du temps d'entrainement.

L'influence de `next_state_dim`

On va ensuite augmenter la dimension des vecteurs après le message-passing, en passant de `next_state_dim = 64` à `next_state_dim = 256`.
Les résultats sont présentés sur la figure III.6 ci-dessous.

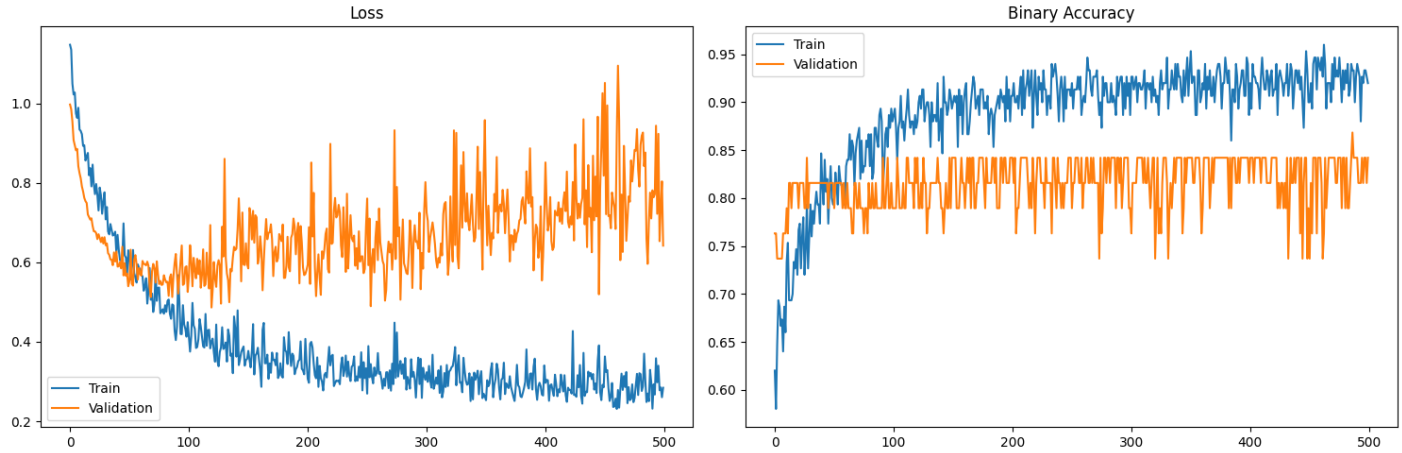


FIGURE III.6 – *Courbes de perte et de précision du modèle avec* `next_state_dim = 256`

- Au-delà de la 119ème époque, le modèle est en surapprentissage. Celui-ci se produit donc plus tardivement que pour le modèle de référence.
  La valeur minimale atteinte par la perte de validation est 0.487, très faiblement moins bonne que le modèle de référence.
- A cette époque, la précision atteint 81.6%, soit identique au modèle de référence.
- Le nombre de paramètres du modèle a très fortement augmenté, atteignant 256401.
  Ce résultat est attendu, car bien que l'augmentation de la dimension après le message-passing augmente le nombre de paramètres, en particulier au cours des étapes de message-passing, qui sont les plus nombreuses dans le modèle, ce qui fait très fortement augmenter le nombre de paramètres.
- L'entrainement de ces 500 époques a duré 2 minutes et 2 secondes, une augmentation très importante par rapport au modèle de référence.

L'influence de `num_message_passing`

On va enfin essayer d'ajouter des étapes de message-passing au modèle, c'est à dire le nombre de voisins qu'on va prendre en considération pour chaque nœud, en modifiant `num_message_passing = 3` à `num_message_passing=6`. Les résultats sont présentés sur la figure III.7 ci-dessous.

On se limite à cette valeur pour éviter, car au-delà, étant donné les tailles relativement limitées des graphes, une valeur plus élevée créerait un "lissage" des valeurs de représentation des nœuds extrêmement important et fausserait complétement les résultats.

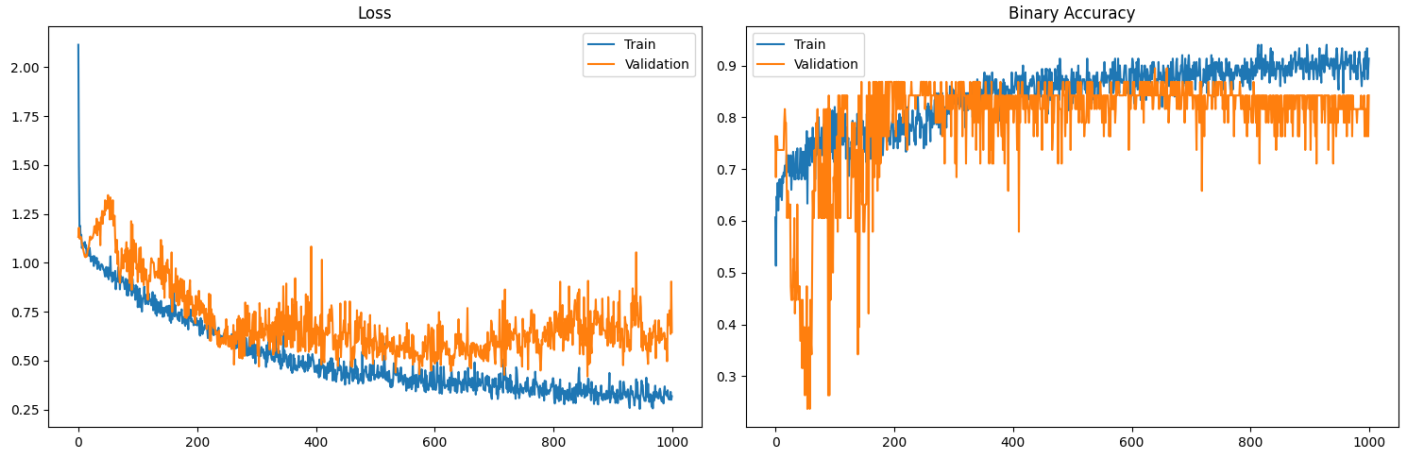


FIGURE III.7 – *Courbes de perte et de précision du modèle avec* `num_message_passing = 6`

- Le modèle atteint le surapprentissage à partir de l'époque 557. Celui-ci se produit donc bien plus tardivement que pour le modèle de référence.
  Cela est cohérent car le nombre plus élevé d'étapes de message-passing augmente la profondeur du modèle en plus du nombre de paramètres. Il faut donc plus d'époques pour que le modèle commence à surapprendre.
  La valeur minimale atteinte par la perte de validation est 0.418, bien meilleure que le modèle de référence.
- A cette époque, la précision atteint 84.2%, supérieure au modèle de référence.
- Le nombre de paramètres du modèle a sensiblement augmenté, atteignant 96273.
- L'entrainement des 500 premières époques a duré 1 minute et 32 secondes, une augmentation importante par rapport au modèle de référence.
  Ce résultat est attendu, car bien que l'augmentation de la profondeur du modèle augmente le nombre de paramètres à ajuster de manière importante.

I-E Résumé de l'influence des hyperparamètres

On peut résumer les influences d'une augmentation des différents hyperparamètres spécifiques aux GNN testés ci-dessus dans le tableau III.1

Hyperaramètre	Temps d'entrainement	Précision atteinte	Délai avant surapprentissage
`node_dim`	Augmentation négligeable	Identique	Légerement plus tardif
`edge_dim`	Augmentation négligeable	Identique	Légèrement plus tardif
`message_dim`	Augmentation importante	Amélioration sensible	Plus tardif
`next_state_dim`	Augmentation très importante	Amélioration sensible	Légèrement plus tardif
`num_message_passing`	Augmentation significative	Amélioration sensible	Bien plus tardif

Table III.1 – Résumé des influences des hyperparamètres spécifiques aux GNN

Concernant les autres hyperparamètres du modèle, non testés ici, on peut s'attendre aux influences suivantes, similaires aux réseaux de neurones classiques :

Hyperaramètre	Temps d'entrainement	Précision atteinte	Délai avant surapprentissage
`dropout_rate`	Identique	Identique	Plus tardif
`l2_regularization`	Identique	Identique	Plus tardif
`steps_per_epoch`	Augmentation linéaire	Identique	Plus tot

Table III.2 – Résumé des influences des hyperparamètres classiques de réseaux de neurones

Enfin, à propos des fonctions d'activation, il existe toute une collection de fonctions d'activation possibles, chacune ayant ses propres caractéristiques. En les modifiant, on pourrait s'attendre à des variations mineures dans les performances du modèle, bien que l'impact exact dépendrait de la nature des données et de la tâche spécifique.

En supposant toutefois que les influences pourraient être similiaires à celles observées dans les réseaux de neurones classiques, on pourra se référer à un de mes précédents travaux sur le sujet. [5]

Pour poursuivre notre étude, il faut remarquer qu'ici, la taille du dataset initial (188 graphes) étant relativement limité, ce qui est pertinent dans une première démarche d'expérimentation sur les concepts. Il est cependant difficile de quantifier précisément l'impact des paramètres sur les performances du modèle, au regard de la taille du dataset. C'est pourquoi on s'est limité ici à une étude qualitative des tendances attendues.

Afin de désormais de vérfier la robustesse des GNN, on va à présent reproduire cette méthode sur un dataset de taille bien plus importante.

II Extension à un dataset plus grand

II-A Présentation du jeu de données

On s'intéresse à nouveau à un problème de classification de graphes, toujours au sujet de la mutagencité des molécules, mais cette fois avec un dataset plus grand : **Mutagenicity** [6].

Celui ci présente désormais 4337 graphes, répartis dans un jeu de données d'entrainement, où les molécules ont, en moyenne, 30.02 atomes et dont 1545 sur 3470 (44.52%) sont mutagènes, et dans un jeu de données de validation où les molécules ont en moyenne 31.49 atomes et où 391 sur 867 (45.10%) sont mutagènes.

Cette fois-ci, les deux ensembles de données présentent une répartition des classes similaire, ce qui est un point important pour l'évaluation des performances du modèle.

Désormais, les nœuds représentent plus de types d'atomes différents :

- Le carbone (C)
- L'oxygène (O)
- Le chlore (Cl)
- L'hydrogène (H)
- L'azote (N)
- Le fluor (F)
- Le brome (Br)
- Le soufre (S)
- Le phosphore (P)
- L'iode (I)
- Le sodium (Na)
- Le Potassium (K)
- Le Lithium (Li)
- Le calcium (Ca)

Les arêtes quant à elles représentent les types de liaisons. On prend désormais en compte :

- Les liaison simples
- Les liaisons doubles
- Les liaisons triples

Les liaisons aromatiques ne sont plus prises en compte dans ce dataset.

De même que précédemment, on peut alors représenter les molécules sous forme de graphe avec le module `Networkx`, comme la figure III.8 le démontre.

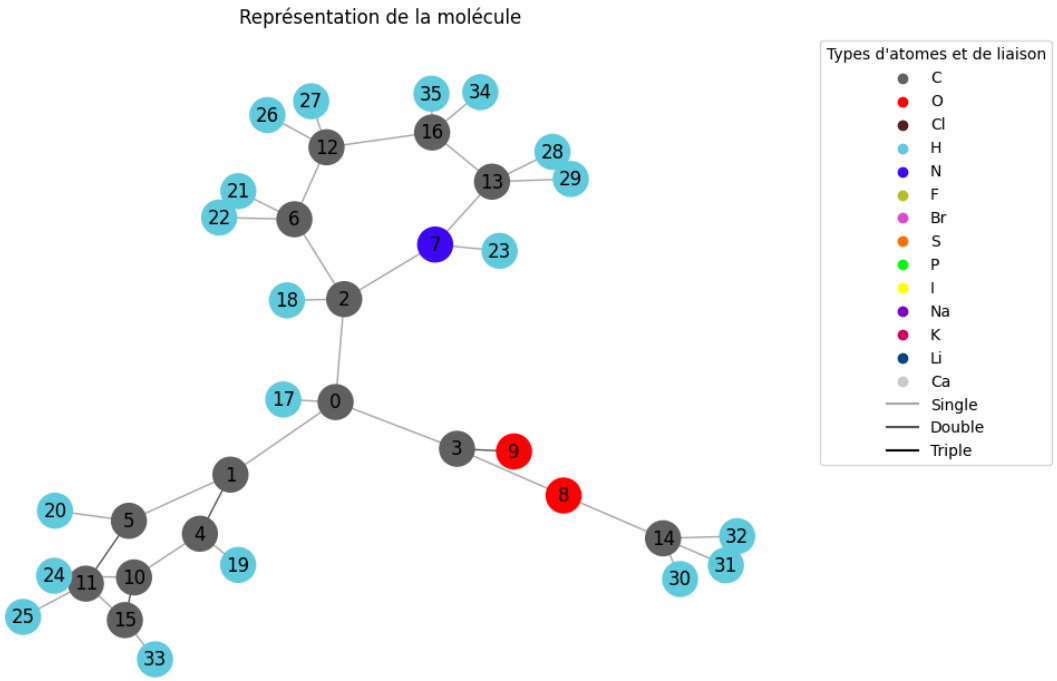


FIGURE III.8 – *Représentation d'une molécule du jeu de données Mutagenicity avec* `Networkx`

Cela nous permet de confirmer que la plupart des molécules sont plus grandes que dans le dataset précédent, avec un nombre de nœuds plus important en moyenne.

II-B Modélisation informatique

On reprend précisément la même modélisation informatique que précédemment.

Le travail de lecture et de mise en forme des données a été effectué dans ce but, voir le code source du projet sur Github pour plus de détails. [7]

Puisqu'on traite le même problème, modélisé de la même manière, on reprend la même infrastructure pour le modèle d'apprentissage.

II-C Utilisation de modèles et analyse des résultats

Modèle de référence

Pour définir un modèle de référence, on choisit les hyperparamètres suivants :

- `node_dim = 16`
- `edge_dim = 16`
- `message_dim = 64`
- `next_state_dim = 64`
- `num_message_passing = 3`
- `activation_function = "relu"`
- `dropout_rate = 0.5`
- `l2_regularization = 5e-4`
- `batch_size = 16`
- `optimizer = Adam(learning_rate=0.01)`
- `steps_per_epoch = 10`

On obtient alors un modèle avec 43761 paramètres.

On obtient après l'entrainement les courbes de pertes et de précision de la figure III.9, sur les données d'entrainement et de validation.

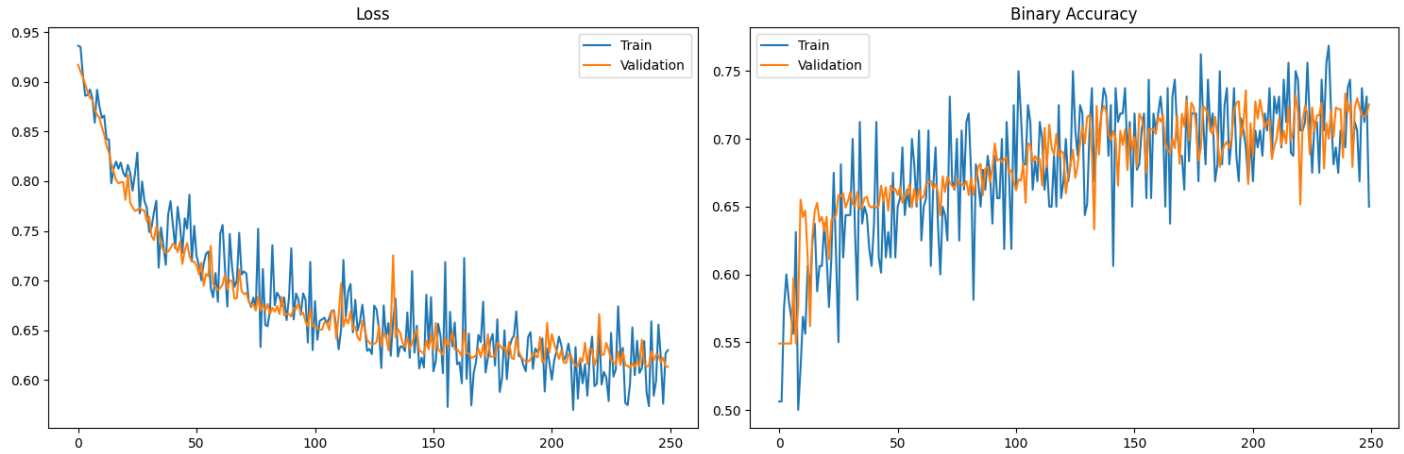


FIGURE III.9 – *Courbes de perte et de précision du modèle de référence sur Mutagenicity*

On accompagne aussi ces courbes des résultats suivants :

- L'entrainement de ces 250 époques à duré 64 minutes et 4 secondes.
- Perte de validation minimale à l'époque 233 pour une précision de 72.09%
- Meilleure précision de validation atteinte : 73.59% à l'époque 197.
- Précision à la dernière époque 72.55% dont la matrice de confusion est donnée sur la figure III.10.

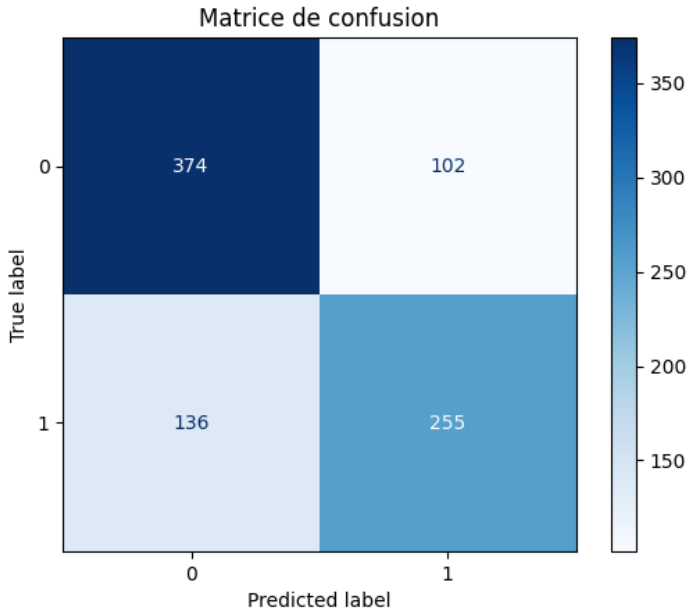


FIGURE III.10 – *Matrice de confusion du modèle de référence*

Remarques :

- Ces valeurs de précision démontrent que ce modèle effectue un apprentissage significatif, car il est bien meilleur que le hasard (50% de précision attendue pour une classification binaire aléatoire).
- Il faut noter que, comme le surapprentissage n'est ici pas atteint, on pourrait espérer améliorer encore les performances du modèle de quelques points en augmentant le nombre d'époques d'entrainement.
- La matrice de confusion de la figure III.10 semble elle démontrer que le modèle n'est pas (ou peu) biaisé puisque les deux types d'erreurs sont relativement équivalentes.

Modèle approfondi

Puisque les graphes sont plus grands dans ce dataset, il serait intéressant de prendre en compte un contexte plus large autour chaque nœud.

On décide pour cela d'augmenter le nombre d'étapes de message-passing, définissant désormais `num_message_passing = 10`. Ce choix définit donc un modèle avec 166513 paramètres dont les courbes de pertes et de précision sont représentées sur la figure III.11 :

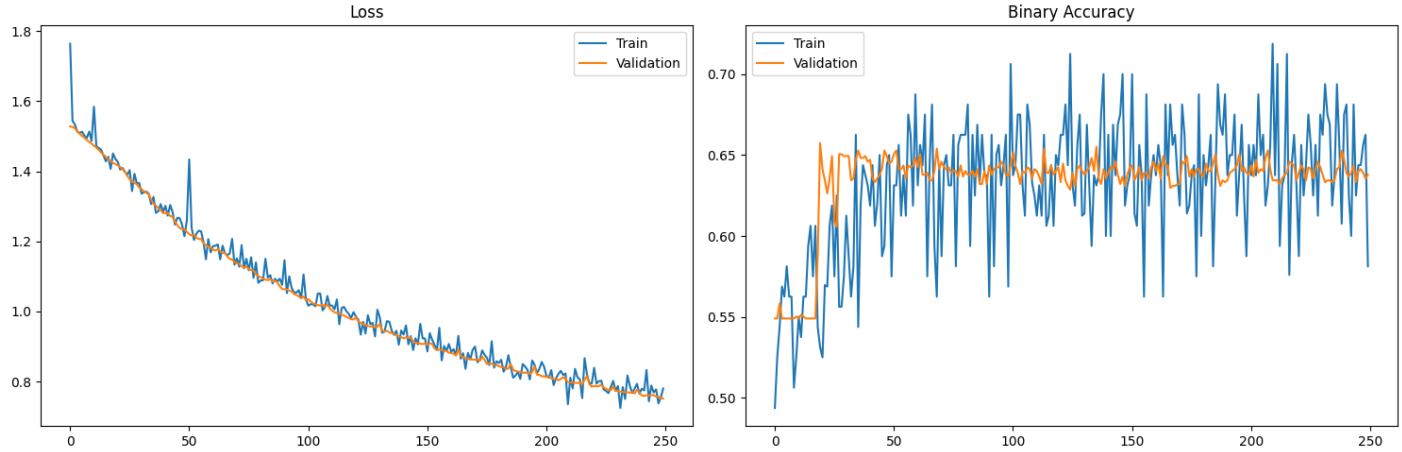


FIGURE III.11 – *Courbes de perte et de précision du modèle approfondi*

- L'entrainement de ces 250 époques à duré 72 minutes et 37 secondes.
- La précision de validation stagne autour de 65% après environ 25 époques.
- La matrice de confusion à la dernière époque est donnée par la figure III.12 :

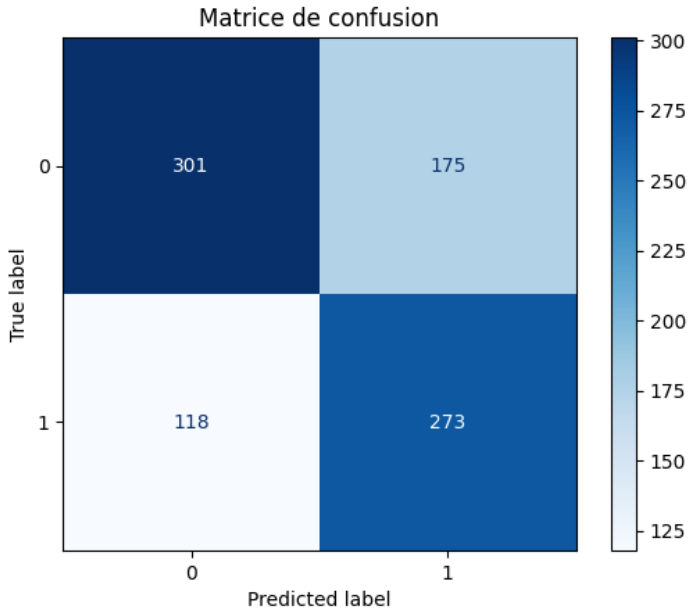


FIGURE III.12 – *Matrice de confusion du modèle approfondi*

Remarques :

- La courbe de précision de validation est ici intrigante. En effet, on observe une stagnation très rapide de la précision, qui ne s'améliore plus par la suite. Plusieurs pistes pourraient expliquer ce phénomène :
  - Le modèle est trop profond pour la taille des graphes, et le message-passing sur 10 étapes crée une boucle d'information sur certains graphes plus petits. Ainsi, le modèle se confronte à une limite maximale de précision.
    Cela implique alors que, pour chaque dataset, il faudrait adapter le nombre d'étapes de message-passing en compromis entre richesse de représentation et éviter la ce problème de taille limitant.
  - Le modèle est ici en sous-apprentissage, du fait du nombre important de paramètres. Il se peut qu'il faille considérablement augmenter le nombre d'époques d'entrainement pour que le modèle puisse ajuster correctement ses paramètres et dépasser ce palier de précision.
- Malgré ce défaut, le modèle réalise malgré tout un apprentissage significatif, car la précision à la dernière époque est de 65.23%, bien meilleure que le hasard.
- Le modèle semble ici légèrement biaisé en défaveur de la classe 0 (non-mutagène).
  Cependant, pour confirmer ce résultat, il faudrait effectuer davantage de prédictions sur des données de validation supplémentaires.

Un modèle avec le Master Node

On veut cette fois prendre en compte un contexte global pour chaque apprentissage de graphe. On ajoute donc cette fois un Master Node au modèle, en plus des 3 étapes de message-passing de référence.

Pour ce faire, voici les étapes de code spécifiquement utilisées dans ce but :

1. On commence à nouveau par recevoir les entrée du graphe, en précisant qu'on reçoit cette fois un contexte global également (initialement nul)

```
def set_initial_node_state(node_set, *, node_set_name):
    return tf.keras.layers.Dense(node_dim)(node_set[tfgnn.HIDDEN_STATE])

def set_initial_edge_state(edge_set, *, edge_set_name):
    return tf.keras.layers.Dense(edge_dim)(edge_set[tfgnn.HIDDEN_STATE])

def set_initial_context_state(context):
    return tf.keras.layers.Dense(next_state_dim)(context[tfgnn.HIDDEN_STATE])

graph = tfgnn.keras.layers.MapFeatures(
    node_sets_fn=set_initial_node_state,
    edge_sets_fn=set_initial_edge_state,
    context_fn=set_initial_context_state
    )(graph)
```

Listing III.6 – Initialisation de tous les états du graphe

2. Ensuite, et en se servant à nouveau des couches `dense` définie dans la sous-section précédente, on construit les étapes de message-passing en mettant à jour les nœuds ainsi le contexte global à chaque étape, et en se servant de ce dernier pour enrichir les messages échangés entre les nœuds :

```
for _ in range(num_message_passing):
    graph = tfgnn.keras.layers.GraphUpdate(
        # Mise a jour des noeuds
        node_sets={
            "atoms": tfgnn.keras.layers.NodeSetUpdate(
                {"bonds": tfgnn.keras.layers.SimpleConv(
                     sender_edge_feature=tfgnn.HIDDEN_STATE,
                     message_fn=dense(message_dim),
                     reduce_type="sum",
                     receiver_tag=tfgnn.TARGET)},
                tfgnn.keras.layers.NextStateFromConcat(
                    dense(next_state_dim)))},

        # Mise a jour du contexte global
        context=tfgnn.keras.layers.ContextUpdate(
            {"atoms": tfgnn.keras.layers.Pool(
                tfgnn.CONTEXT,
                reduce_type="mean",
                node_set_name="atoms")},
            tfgnn.keras.layers.NextStateFromConcat(dense(next_state_dim)))
        )(graph)

    # Prise en compte du contexte global dans la mise a jour des noeuds
    context_state = graph.context[tfgnn.HIDDEN_STATE]
    node_sizes = graph.node_sets["atoms"].sizes
    broadcasted_context = tf.repeat(context_state, repeats=node_sizes, axis=0)

    graph = graph.replace_features(
    node_sets={
        "atoms": {
            tfgnn.HIDDEN_STATE: tf.concat(
                [graph.node_sets["atoms"][tfgnn.HIDDEN_STATE],
                   broadcasted_context],
                axis=-1)}})
```

Listing III.7 – Message-passing avec Master Node

3. Enfin, on agrège les représentations finales des nœuds de chaque graphe du batch avec la représentation globale finale du graphe et une couche dense finale pour prédire la classe, exactement comme précédemment.
4. On définit également les mêmes fonction de perte (Entropie croisée binaire) et de métrique (Précision binaire) que précédemment.

On obtient cette fois un modèle avec 93297 paramètres dont les courbes d'entrainement et de validation se trouve sur la figure III.13 :

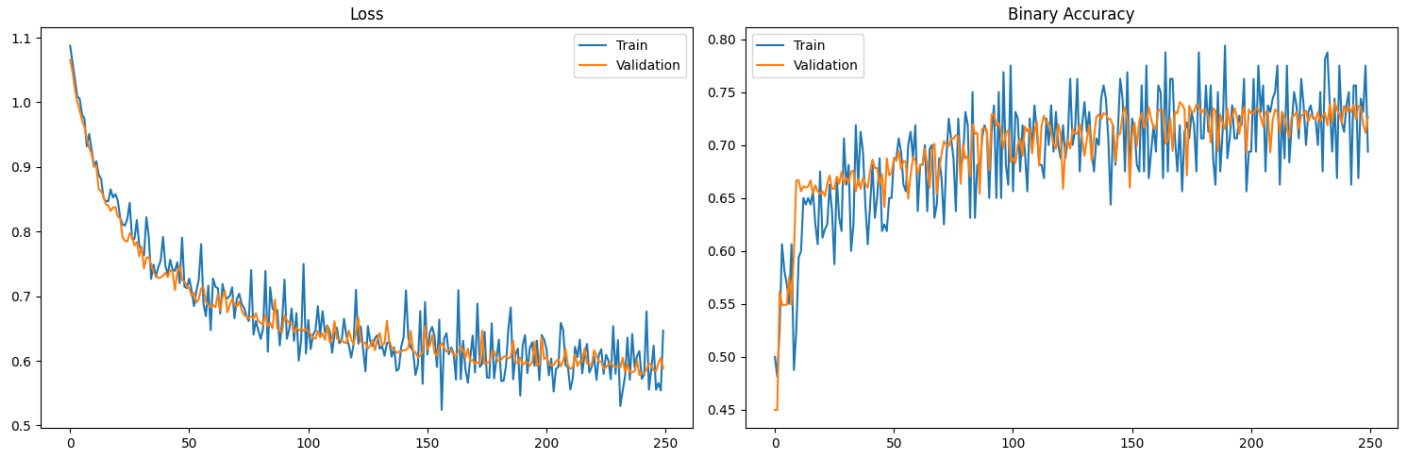


FIGURE III.13 – *Courbes de perte et de précision du modèle avec Master Node*

- L'entrainement de ces 250 époques à duré 68 minutes et 23 secondes.
- Perte de validation minimale à l'époque 239 pour une précision de 73.70%
- Meilleure précision de validation atteinte : 74.04% à l'époque 170.
- Précision à la dernière époque 72.66% dont la matrice de confusion est donnée sur la figure III.14.

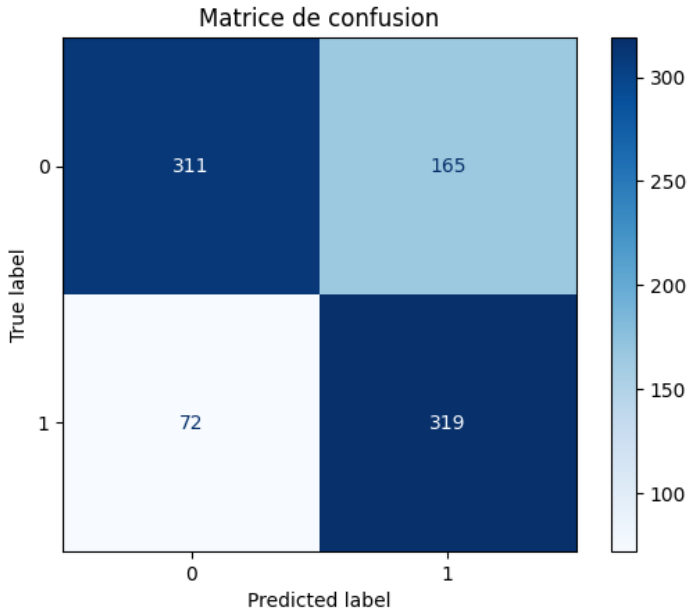


FIGURE III.14 – *Matrice de confusion du modèle avec Master Node*

Remarques :

- Les performances de ce modèle sont légèrement meilleures que celles du modèle de référence, ce qui démontre que la prise en compte d'un contexte global par le Master Node apporte un plus dans la qualité de l'apprentissage.
- Cette amélioration reste toutefois modeste ici, ce qui peut s'expliquer par le fait que les tailles des graphes du dataset restent malgré tout raisonnable. Ainsi, le message-passing sur 3 étapes permet déjà de capturer une grande partie des relations entre nœuds, rendant la prise en compte d'un contexte global moins cruciale.
- Il se peut, ici aussi, que le modèle soit en léger sous-apprentissage, et qu'une augmentation du nombre d'époques d'entrainement permette d'améliorer davantage les performances de quelques points.
- Enfin, ici aussi le modèle semble légèrement biaisé en défaveur de la classe 0 (non-mutagène).
  A nouveau, pour confirmer ce résultat, il faudrait effectuer davantage de prédictions sur des données de validation supplémentaires.

II-D Récapitulatif des résultats

On peut désormais résumer les performances quantitatives des différents modèles testés sur le dataset Mutagenicity dans le tableau ci-dessous :

Modèle utilisé	Temps d'entrainement	Précision maximale atteinte
3 message-passing sans Master Node	64m 03s	73.59%
10 message-passing sans Master Node	72m 37s	65.03%
3 message-passing avec Master Node	68m 23s	74.04%

Table III.3 – Résumé des performances des modèles sur Mutagenicity

Conclusion

Pour conclure, les Graph Neural Networks (GNN) représentent une avancée significative dans le domaine de l'apprentissage automatique sur les données structurées en graphes. Leur capacité à capturer les relations complexes entre les nœuds et les arêtes permet de résoudre efficacement des problèmes variés, allant de la classification de graphes à la prédiction de liens.

Cependant, la conception et l'entraînement de GNN efficaces nécessitent une compréhension approfondie des concepts sous-jacents, afin de prendre en considération la connectivité induite par cette structure de données. Les éléments les plus importants à retenir sont les suivants :

- La représentation des graphes pour une utilisation dans un GNN ne peut pas se faire correctement avec la matrice d'adjacence. Il est nécessaire d'utiliser la liste d'adjacence pour capturer de manière efficace et unique la structure du graphe.
- Les GNN utilisent le message-passing pour permettre aux nœuds d'échanger des informations avec leurs voisins, ce qui permet de capturer les relations locales dans le graphe. Ces échanges d'informations locales peuvent être effectués, entre autres méthodes, par des convolutions sur graphes, inspirées des convolutions sur les images.
- Pour les très grands graphes, le message-passing ne suffit pas à capturer les informations globales. Il est alors nécessaire d'utiliser une représentation globale dédiée, appelée le *Master Node*. Cette représentation peut être obtenue, entre autres méthodes, par des convolutions spectrales, qui prennent en compte la structure globale du graphe en se servant de la théorie de Fourier.
- Toutes ces particularités introduisent alors de nouveaux hyperparamètres spécifiques aux GNN, qui influencent les performances du modèle. Il est important de comprendre l'influence de ces hyperparamètres pour optimiser les performances des GNN sur des tâches spécifiques.

En somme, les GNN offrent un cadre puissant pour l'apprentissage automatique sur les données en graphes.

Perspectives

Le sujet est vaste, tout comme ses applications et les perspectives ne manquent pas en ce qui concerne les poursuites envisageables. Parmi les pistes possibles, on peut citer :

- L'application des méthodes de convolutions sur graphes plus avancées, comme les convolutions ChebNet ou les convolutions spectrales, pour en vérifier respectivement l'amélioration de stabilité et la prise en compte d'un contexte global.
- Une comparaison plus approfondie entre les GNN et d'autres architectures de réseaux de neurones, comme les réseaux de neurones récurrents (RNN) ou les transformers, pour évaluer leurs performances respectives sur des tâches spécifiques.
- La réalisation de tâches de classification sur les nœuds ou les arêtes, plutôt que sur les graphes entiers.
- L'ouverture à d'autres sujets d'applications que les molécules, comme les réseaux sociaux, les systèmes de recommandation, les réseaux de transport, etc...

Bibliographie

[1] B. Sanchez-Lengeling, E. Reif, A. Pearce, A. Wiltschko (2021). *A gentle Introduction to Graph Neural Networks.*
https://doi.org/10.23915/distill.00033

[2] A. Daigavane, B. Ravindran, G. Aggarwal (2021). *Understanding Convolutions on Graphs.*
https://doi.org/10.23915/distill.00032

[3] T. N. Kipf, M. Welling (2017). *Semi-Supervised Classification with Graph Convolutional Networks.*
https://arxiv.org/abs/1609.02907

[4] Exemple d'utilisation de Tensorflow-GNN sur Mutag
https://github.com/tensorflow/gnn/blob/main/examples/notebooks/intro_mutag_example.ipynb

[5] M. Speismann, J.St-Germain (2025). *Impact des fonctions d'activation dans les réseaux de neurones profonds.*
(Rapport réalisé dans le cadre d'un projet pédagogique à Polytechnique Montréal)
https://drive.google.com/file/d/1rzBzkZchuQ5xJBAjIhqXOI46K4qGLIGs/view?usp=sharing

[6] Source du dataset Mutagenicity
https://networkrepository.com/Mutagenicity.php

[7] Code source du projet
https://github.com/Matthieu-Speismann/Projet_3A_GNN

[8] Y-C Hsiao (2024). *Derivation of Back-propagation for Graph Convolutional Networks using Matrix Calculus and its Application to Explainable Artificial Intelligence*
https://arxiv.org/html/2408.01408v1?.com

Pour en savoir plus sur la rétropropagation des erreurs dans les GNN avec l'utilisation de convolutions sur graphes